

Notas de Clase (05 de junio de 2026): Flutter y pub.dev

Resumen Completo de Programación III (Clase del 05/06/2026)

Notas de Clase: Flutter

El ecosistema de pub.dev

pub.dev es el repositorio oficial de paquetes para Flutter. Permite integrar herramientas de terceros como just_audio para reproducir sonidos en la aplicación.

Librerías Destacadas

google_nav_bar: Paquete para crear barras de navegación inferiores animadas con efecto de pastilla expandible. font_awesome_flutter: Biblioteca con miles de iconos vectoriales de alta calidad, ideales para redes sociales.

Archivo pubspec.yaml y Dependencias

pubspec.yaml es el archivo de configuración donde se listan las dependencias bajo la sección dependencies. Es altamente sensible a la sangría, por lo que se deben usar espacios y NUNCA el tabulador. Al guardar, se ejecuta flutter pub get automáticamente.

Extension Pubspec Assist

Herramienta de VS Code para agregar dependencias desde la paleta de comandos (Ctrl + Shift + P), evitando errores manuales de formato.

Arquitectura de Rutas Centralizada

Consiste en usar un archivo como app_routes.dart para centralizar las rutas de la aplicación de forma escalable. Usar listas dinámicas como menuOptions permite estructurar las opciones del menú con su ruta, nombre e icono para dibujar interfaces automáticamente.

Uso de Font Awesome

Al buscar iconos, se debe activar el filtro Free para evitar los de pago. En el código de Flutter, los nombres se transforman a camelCase (ejemplo: box-open pasa a ser FontAwesomeIcons.boxOpen).

Aplicacion Practica de Font Awesome:

El profesor actualizo el archivo `app_routes.dart` para reemplazar los iconos nativos de Material (`Icons.*`) por los del paquete `font_awesome_flutter` (`FontAwsomelcons.*`). Se agrego una nueva ruta `'/dep'` para la Pantalla de Dependencias, utilizando precisamente el icono `FontAwsomelcons.boxOpen` que se busco en la web, demostrando como mejorar la interfaz visual de la lista generada automaticamente.

Implementacion de `google_nav_bar`:

El profesor comenzo a programar la barra animada en la nueva pantalla (`dependencies_screen.dart`). Para ello, asigna el widget `GNav` (del paquete `google_nav_bar`) a la propiedad `bottomNavigationBar` del `Scaffold`. Dentro de `GNav`, se utiliza la propiedad `tabs` que recibe una lista de widgets `GButton`. Cada `GButton` representa una pestaña u opcion del menu animado y requiere obligatoriamente el parametro `icon`.

Personalizacion y Resultado de `GNav` (`google_nav_bar`):

El profesor completo la estructura de la barra agregando varios `GButton` (como Inicio, Busqueda y Perfil). En el emulador ya se puede ver el resultado final con el efecto visual de "pastilla" en la pestaña activa ("Menu principal"). Ademias, se resalto como el editor (VS Code) muestra mediante el autocompletado todas las propiedades que tiene `GNav` (`gap`, `padding`, `activeColor`, `tabBackgroundColor`, etc.), lo cual es vital para personalizar los colores y espacios de la barra sin tener que memorizar el codigo.

Fundamentos de Layout: El Widget Container

El profesor hizo una pausa para explicar las bases para maquetar espacios, usando el widget `Container` (reemplazando temporalmente la barra animada para demostrarlo en el `bottomNavigationBar`). Para dimensionar una caja, se definen tres propiedades clave: alto (`height`), ancho (`width`) y color. Se destaco un truco muy pro: el uso de `"width: double.infinity"`, lo cual obliga al contenedor a estirarse y ocupar absolutamente todo el ancho disponible de la pantalla, sin importar el tamaño del celular. Es una tecnica ideal para probar espacios antes de meter los botones definitivos.

Manejo de Inputs y Teclados (Entradas de Texto):

El profesor avanzo al tema de recoleccion de datos creando una nueva pantalla llamada `inputs_screen.dart`. Para integrarla al sistema de navegacion, actualizo el archivo `app_routes.dart` agregando la ruta `inputs` y añadiendo una nueva opcion en la lista `menuOptions` con el nombre "Pantalla de Inputs/Entradas", utilizando el icono `FontAwesomeIcons.solidKeyboard`. Esto sienta las bases para trabajar con campos de texto (`TextFields`) y formularios en la aplicacion.

Prevencion de Desbordamiento (`SingleChildScrollView`):

Para preparar la pantalla de inputs, el profesor explico como evitar errores de overflow (cuando el contenido supera el tamaño de la pantalla, un problema muy comun al abrir el teclado del celular). La solucion es envolver la Column principal de la pantalla dentro de un widget llamado `SingleChildScrollView`. El profesor lo demostro apilando decenas de textos 'Hola mundo' para visualizar como la pantalla adquiere la capacidad de hacer scroll (deslizarse hacia arriba y abajo) automaticamente.

Implementacion de Formularios (`Form` y `TextFormField`):

El profesor borro los textos de prueba y armo la estructura real para capturar datos. Introdujo el widget `Form`, que funciona como un contenedor principal para agrupar, gestionar y validar multiples campos de texto al mismo tiempo. Dentro de este, colocho el widget `TextFormField()`, que es el cuadro interactivo donde el usuario ingresa la informacion. En el emulador se evidencia que, al interactuar con el `TextFormField`, el teclado se despliega correctamente sin causar errores visuales, gracias a la implementacion previa del `SingleChildScrollView`.

Configuracion de `TextFields` y Selector de Fechas (`DatePicker`):

El profesor agrego `Padding` al formulario para darle margenes a los lados y mejorar el diseño. En el `TextFormField` enseño dos propiedades clave: `"autofocus: true"` (obliga al teclado a abrirse

automaticamente apenas entras a la pantalla) y `"keyboardType: TextInputType.datetime"` (cambia el formato del teclado para mostrar directamente los numeros). Finalmente, introdujo el widget `DatePickerDialog`, el cual despliega un calendario interactivo nativo, donde se pueden limitar los rangos de seleccion usando las propiedades `firstDate` y `lastDate` (ejemplo: 2027 a 2030).

Decoración de TextFields (InputDecoration):

El profesor continuó personalizando el `TextFormField` para mejorar la experiencia del usuario. Cambió el tipo de teclado a texto normal con `"keyboardType: TextInputType.text"`. Explicó la propiedad `"obscureText"` (que se ve comentada en el código), la cual es fundamental para ocultar los caracteres ingresados, ideal para campos de contraseñas. Finalmente, introdujo la propiedad `"decoration"` utilizando el widget `InputDecoration`, aplicando un `"helperText"` ('Hey, solo letras'). Esto añade un pequeño mensaje de asistencia debajo de la línea principal para guiar al usuario sobre qué formato de texto debe ingresar.

Iconos Interactivos en TextFields (suffixIcon):

Dentro de la propiedad `decoration`, el profesor agregó un `"suffixIcon"`. Esto permite incrustar un widget interactivo al final (lado derecho) del campo de texto. En el ejemplo de la clase, usó un `IconButton` con el ícono de un ojo (`Icons.remove_red_eye`). Esta es la técnica estándar y más utilizada en el desarrollo móvil para crear el botón de "mostrar/ocultar contraseña" en los formularios de inicio de sesión o registro.

Captura de Datos y Diseño Avanzado de TextFields:

El profesor agregó la propiedad `"onChanged"` al `TextFormField`, lo cual permite capturar y leer en tiempo real cada letra que el usuario va escribiendo (se demostro imprimiendo el valor en consola con un `print`). Además, profundizó en el diseño visual agregando `"hintText"` (texto de marca de agua o placeholder que dice 'Email') y `"labelText"` (la etiqueta principal que flota al seleccionar el campo, 'Escribe tu email'). También mostro como personalizar el recuadro del input cuando esta activo (enfocado) usando la propiedad `"focusedBorder"` junto con `"OutlineInputBorder"` y `"BorderSide"`, lo que permite cambiar el color y grosor de la linea del

cuadro.

Bordes Personalizados y Control de Versiones (Git):

El profesor finalizó el diseño del input personalizando el borde estático usando la propiedad "border" con un "OutlineInputBorder". Mostró cómo hacer bordes asimétricos usando "borderRadius: BorderRadius.only", redondeando específicamente solo algunas esquinas (bottomLeft y topRight a 10) para darle un estilo único de "hoja" al cuadro de texto. Además, en el panel izquierdo de VS Code, abrió la pestaña de Control de Código Fuente (Git), mostrando todos los archivos modificados (con la letra M), lo que indica que está preparando un "commit" para guardar, registrar y respaldar todo el código escrito en la clase.